

Tópicos Especiais em Inteligência Computacional Aplicada a Segurança de Sistemas

Aula 05

Programa de Pós-graduação em Computação Aplicada
Núcleo de Desenvolvimento Amazônico em Engenharia
Prof. Dr. Caio C. Moreira

Ciclo de um sistema baseado em ML aplicado à segurança

- Coleta de dados
 - Logs, tráfego de rede, arquivos executáveis, eventos de execução.
 - Bases públicas de malwares (amostras brutas).
 - Bases Públicas com Características Extraídas (datasets prontos para ML)
- Pré-processamento
 - Limpeza e normalização dos dados.
 - Conversão de logs/arquivos em representações numéricas.
 - Tratamento de classes desbalanceadas (SMOTE, undersampling).
- Extração de características (feature engineering)
 - Permissões solicitadas por aplicativos.
 - Chamadas de API/sistema.
 - Frequência de pacotes em tráfego de rede.
 - Strings e entropia em executáveis.
- Treinamento do modelo
 - **Escolha do tipo de algoritmo (supervisionado ou não supervisionado).**
 - **Escolha do modelo**
 - **Ajuste de hiperparâmetros.**
- Avaliação do modelo
 - Divisão em treino/teste (ou validação cruzada).
 - Métricas: Acurácia, Precisão, Recall, F1-score, AUC-ROC.
- Interpretação de resultados para validar a utilidade prática.

Treinamento do Modelo

- O objetivo do treinamento é ensinar o modelo a **reconhecer** padrões e comportamentos nos dados.
 - O modelo deve ser capaz de aprender padrões e realizar previsões a partir dos dados, minimizando erros e generalizando bem para novos exemplos.

Treinamento do Modelo

- **Escolha do tipo do algoritmo (supervisionado ou não supervisionado).**
- A escolha depende da natureza dos dados e do objetivo da análise.
 - Aprendizado supervisionado:
 - O modelo aprende com exemplos rotulados (dados de entrada + saída esperada).
 - Usado para classificação (malware vs. benigno) ou regressão (valores contínuos).
 - Exemplos: Random Forest, XGBoost, SVM, Redes Neurais.
 - Aprendizado não supervisionado:
 - Não há rótulos. O modelo busca padrões ocultos ou agrupamentos nos dados.
 - Usado para detecção de anomalias ou agrupamento de comportamentos similares.
 - Exemplos: K-Means, DBSCAN, Autoencoders.

Treinamento do Modelo

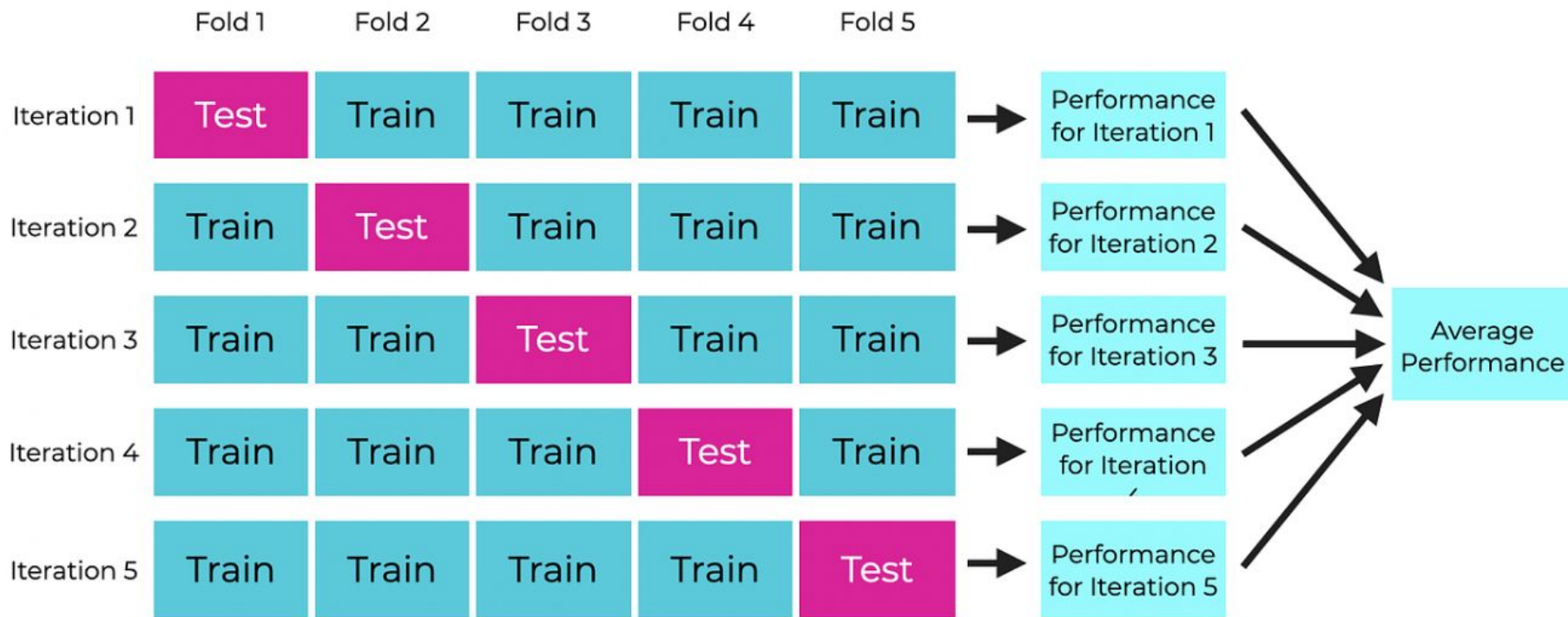
- **Supervisionado**

- **Com Validação Cruzada**

- Técnica para **avaliar** o desempenho real de um modelo de aprendizado de máquina.
 - Evita que o modelo seja avaliado apenas com base nos mesmos dados usados no treinamento (o que gera overfitting).
 - Como funciona:
 - O conjunto de dados é dividido em K partes (folds) de tamanho semelhante.
 - O modelo é treinado em $K-1$ partes e testado na parte restante.
 - O processo se repete K vezes, trocando o fold de teste a cada iteração.
 - O desempenho final é a média das métricas obtidas em todas as iterações.

Treinamento do Modelo

- **Supervisionado**
 - **Com Validação Cruzada**



Treinamento do Modelo

- **Antes de começar... Carregando e Embaralhando o dataset**

```
import pandas as pd
df_headers = pd.read_csv("../Datasets/headers_pefile.csv")
```

```
from sklearn.utils import shuffle
import numpy as np
```

```
df_headers = shuffle(df_headers)
X = df_headers.iloc[:,4:]
y = df_headers.iloc[:,2]
```

Treinamento do Modelo

- **Supervisionado**
 - **Com Validação Cruzada**
 - **Exemplo:**

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_validate
from sklearn.naive_bayes import BernoulliNB
```

```
num_folds = 10
```

```
kfold = RepeatedStratifiedKFold(n_splits=num_folds, n_repeats=10)
```

```
# X = características (features) das amostras
```

```
# y = respectivos rótulos
```

```
cross_validate(BernoulliNB(), X, y, cv=kfold, scoring=metrics, verbose=0)
```


Treinamento do Modelo

- **Não Supervisionado**

- **Dataset sem rótulos.**
- **Modelo identifica clusters de comportamento anômalo, possivelmente malicioso.**

```
from sklearn.cluster import KMeans
```

```
# Não supervisionado
```

```
kmeans = KMeans(n_clusters=2).fit(X)
```

```
kmeans.predict(X)
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score(y, kmeans.predict(X))
```

Treinamento do Modelo

- **Ajuste de hiperparâmetros**

- Hiperparâmetros são configurações externas do modelo, definidas antes do treino (ex.: número de árvores, taxa de aprendizado, profundidade máxima).
- O ajuste busca o equilíbrio entre desempenho e generalização.

- Exemplos de hiperparâmetros:
 - Random Forest → `n_estimators`, `max_depth`.
 - XGBoost → `learning_rate`, `n_estimators`.
 - SVM → `C`, `kernel`.

Treinamento do Modelo

- **Ajuste de hiperparâmetros**

- Métodos de Ajuste
 - Grid Search: testa todas as combinações possíveis (exaustivo, mas preciso).
 - Random Search: escolhe combinações aleatórias (mais rápido).
 - Bayesian Optimization ou AutoML: ajustam de forma inteligente com base em resultados anteriores.
- Exemplo com Naive Bayes e GridSearch:

```
from sklearn.model_selection import GridSearchCV
model = BernoulliNB()
param_grid = {
    "alpha": [0.1, 0.5, 1.0], "binarize": [None, 0.3, 0.5, 0.7], "fit_prior": [True, False] }
grid = GridSearchCV(
    estimator=model, param_grid=param_grid, scoring="accuracy", cv=5, verbose=1 )

grid.fit(X, y)
print(grid.best_params_)
```